Indian Institute of Technology Jodhpur

Autonomous Systems

A Local Multi-Agent Academic Helpdesk for IIT Jodhpur

Retrieval-Augmented Question Answering, Course Lookup and Email Drafting with Small Local Language Models

Assignment 2 – Project Report

Sagar Ratna Chaudhary
Roll No. B23CM1034

Satyam Jha
Roll No. B23CS1066

Source code:
https://github.com/sagar7162/Autonomous-Assignment-2

October 29, 2026

Abstract

Students at a large technical institute routinely need answers to questions that are scattered across regulation PDFs, curriculum documents, academic calendars, timetable spreadsheets and circular amendments. Finding the correct and *currently applicable* rule is slow, and generic chatbots are unreliable because they answer from parametric memory and cannot cite institutional sources.

This report describes the design, implementation and evaluation of the *IIT Jodhpur Academic Helpdesk*, a fully local multi-agent system. An orchestrator classifies each request and delegates it to one of three specialist roles: a *policy agent* that answers questions from institutional documents using retrieval-augmented generation (RAG) with citations, a *schedule agent* that performs deterministic lookups in CSV timetable and instructor records, and an *email agent* that drafts sick-leave and makeup-exam emails for student review without ever sending them. All language-model inference and embedding is performed locally through Ollama (Qwen3 1.7B for chat and `nomic-embed-text` for embeddings), and documents are indexed in a persistent Chroma vector store. A single-agent mode sharing the same tools allows a controlled comparison with the multi-agent mode.

The system emphasises *trustworthiness over coverage*. It asks for clarification when the programme or admission year is missing, refuses non-academic questions, answers "I don't know based on the available academic records" when evidence is absent, logs every unanswered question, validates citations, applies amendment-aware retrieval priority, and uses placeholders instead of inventing names, dates or documents in drafted emails. We evaluate the system on 25 prompts spanning policy answers, clarification, missing information, email drafting and out-of-scope refusal, and discuss failure modes and limitations honestly. The report documents the architecture, the code-level safeguards and a reproducible evaluation protocol.

Contents

1 Introduction **1**
- 1.1 Motivation . . . 1
- 1.2 Objectives . . . 1
- 1.3 Summary of Contributions . . . 1

2 Background and Related Concepts **1**
- 2.1 Retrieval-Augmented Generation . . . 1
- 2.2 Dense Embeddings and Vector Search . . . 2
- 2.3 Local Language Models with Ollama . . . 2
- 2.4 Multi-Agent Systems . . . 2

3 Problem Statement and Requirements **2**
- 3.1 Functional Requirements . . . 2
- 3.2 Non-Functional Requirements . . . 2

4 System Architecture **3**
- 4.1 Overview . . . 3
- 4.2 Component Responsibilities . . . 3
- 4.3 Repository Structure . . . 3
- 4.4 Multi-Agent versus Single-Agent Mode . . . 4
- 4.5 Technology Stack . . . 4

5 Data and Corpus **4**
- 5.1 Document Corpus . . . 4
- 5.2 Supported Formats . . . 5
- 5.3 Handling Scanned Documents with OCR Sidecars . . . 5
- 5.4 Structured Records . . . 5

6 Ingestion Pipeline **5**
- 6.1 Overview . . . 5
- 6.2 Chunking . . . 5
- 6.3 Embedding . . . 6

6.4 Atomic Index Replacement 6

7 Retrieval **6**
7.1 Baseline Dense Retrieval 6
7.2 Calendar-Aware Filtering 6
7.3 Amendment-Priority Retrieval 6
7.4 Evidence Formatting 7

8 Agent Design **7**
8.1 Chat Interface to the Model 7
8.2 Orchestrator and Routing 7
8.3 Policy Agent 7
8.4 Schedule Agent 8
8.5 Email Agent 8

9 Safeguards and Reliability Mechanisms **9**
9.1 The Attendance Amendment Guard 9
9.2 Unanswered-Question Log 9

10 Implementation Details **9**
10.1 Setup and Usage 9
10.2 Design Decisions and Trade-offs 10
10.3 Testing 10

11 Evaluation Methodology **10**
11.1 Question Set 10
11.2 Reference Expectations 11
11.3 Non-Policy Questions 11
11.4 Evaluation Protocol 11
11.5 Metrics 12

12 Results and Discussion **12**
12.1 Code-Derived Expected Behaviour per Category 12
12.2 Measured Results 13
12.3 Discussion 13

13 Limitations and Failure Analysis **14**

14 Conclusion **15**

1 Introduction

1.1 Motivation

Academic life at an institute such as IIT Jodhpur is governed by a large body of formal documents: programme curricula, the academic regulations, amendments and circulars to those regulations, academic calendars for the autumn, spring and summer terms, examination policies, and timetables that change every semester. A student asking a seemingly simple question, such as "what is the minimum attendance required to sit the major examination?", may need to consult the original regulation *and* a later amendment that modifies it. The cost of consulting the wrong document is not trivial: it can affect registration, examinations and eventually graduation.

Large language models (LLMs) can answer natural-language questions fluently, but they have two properties that make them unsuitable for this setting when used naively. First, they answer from parametric memory and may confidently produce rules that never existed at the institute. Second, they offer no provenance: a student cannot verify where an answer came from. A helpdesk for academic regulation therefore needs grounding, citation and a disciplined way to say "I do not know".

1.2 Objectives

The project targets the following objectives.

1. Build an academic helpdesk that answers policy, curriculum, registration, examination and calendar questions **using only institutional documents**, with citations to the source and page.
2. Support **course information lookup** (slots, timetables, instructors) from structured records.
3. **Draft emails** for common academic requests (sick leave, makeup examination) for the student to review, never sending anything automatically.
4. Organise the functionality as a **multi-agent system** coordinated by an orchestrator, and provide a single-agent baseline for comparison.
5. Run **entirely locally** on modest hardware, without calling external LLM APIs, to preserve privacy and avoid cost.
6. Establish a **reproducible evaluation** with questions that probe not only correct answers but also correct clarification, refusal and abstention.

1.3 Summary of Contributions

- An end-to-end local RAG pipeline supporting PDF, HTML, Word and Excel sources, including hash-verified OCR sidecars for scanned PDFs.
- An orchestrator–specialist architecture with an LLM-based router, rule-based clarification and deterministic tool use for structured data.
- A set of code-level guardrails that make small-model behaviour safer: clarification rules, citation validation, amendment-priority retrieval, an attendance-answer check, email post-processing and unanswered-question logging.
- An atomic re-indexing procedure that preserves the previous index if a build fails.
- A 25-question evaluation set with reference expectations, a reproducible runner, and a separate comparison of two local models.
- A candid analysis of limitations, including routing errors and incomplete coverage.

2 Background and Related Concepts

2.1 Retrieval-Augmented Generation

Retrieval-augmented generation combines a retriever with a generator. Given a question q, the retriever returns the k most relevant text passages $\{d_1, \ldots, d_k\}$ from a corpus, and the generator conditions on both the question and these passages:

$$p(a \mid q) \approx \sum_{d \in \text{top-}k(q)} p_{\text{ret}}(d \mid q)\, p_{\text{gen}}(a \mid q, d). \tag{1}$$

In practice, and in this project, the passages are simply concatenated into the prompt, and the model is instructed to answer only from them. RAG reduces hallucination, allows the knowledge base to be updated without retraining, and enables citation, because each passage carries metadata about its origin.

2.2 Dense Embeddings and Vector Search

Passages and queries are mapped to vectors by an embedding model. Relevance is measured with cosine similarity,

$$\mathrm{sim}(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\| \, \|\mathbf{v}\|}, \tag{2}$$

and the vector store returns the nearest neighbours using an approximate index (HNSW in Chroma). The project uses `nomic-embed-text`, which expects task prefixes: documents are embedded with the prefix `search_document:` and queries with `search_query:`. Using the correct prefixes matters because the model is trained with them to align queries and documents.

2.3 Local Language Models with Ollama

Ollama serves open-weight models through a local HTTP API. The helpdesk calls its `chat` and `embed` endpoints. The default chat model is Qwen3 1.7B, a very small model chosen so that the system runs on ordinary laptops. Small models are fast and private but weaker at instruction following, which motivates the heavy use of deterministic code around them: wherever an answer can be obtained by exact lookup or a rule, the system does not ask the model.

2.4 Multi-Agent Systems

An agent perceives a request, decides on an action, uses tools and produces a result. In a multi-agent design, responsibilities are divided among specialised agents coordinated by an orchestrator. Benefits include separation of concerns, role-specific prompts and the ability to attach role-specific tools. In this project the agents are *roles within one Python process* that share the same local chat model; the specialisation lies in prompts, tools and data sources rather than in separate models or services. This keeps deployment simple while still allowing a meaningful comparison against a single-agent mode.

3 Problem Statement and Requirements

3.1 Functional Requirements

FR1. Answer academic policy, curriculum, registration, examination and calendar questions with citations to a document title, page number and URL.

FR2. Look up course slots, timetable entries and instructors from structured records, only when the semester is explicit and a source URL exists.

FR3. Draft sick-leave and makeup-exam emails for student review; never send email.

FR4. Ask the student for missing context (programme, admission year) when the applicable rule depends on it.

FR5. Refuse out-of-scope, non-academic questions.

FR6. Log every unanswered question with a reason for later curation.

FR7. Provide single-agent and multi-agent operating modes.

FR8. Support PDF, HTML, Word and Excel sources, with OCR support for scanned PDFs.

3.2 Non-Functional Requirements

NFR1. Locality and privacy: no external LLM calls; all processing on the user's machine.

NFR2. Reproducibility: documented commands, fixed temperature (zero), fixed context length, evaluation runner and review artefacts.

NFR3. Robustness: failed re-indexing must not destroy the existing index.

NFR4. Transparency: a live activity trace of which agent is doing what, switchable off with a `-quiet` flag.

NFR5. Modest resources: runnable with a 1.7B-parameter model and a 2,048-token context.

4 System Architecture

4.1 Overview

Figure 1 shows the overall architecture. A student question enters through the command-line interface (`ask` for one-shot use, `chat` for an interactive loop). The orchestrator first applies a rule-based check for missing context, then asks the language model to classify the request into one of four labels: `POLICY`, `SCHEDULE`, `EMAIL` or `OUT`. Each label maps to a specialist or to a refusal.

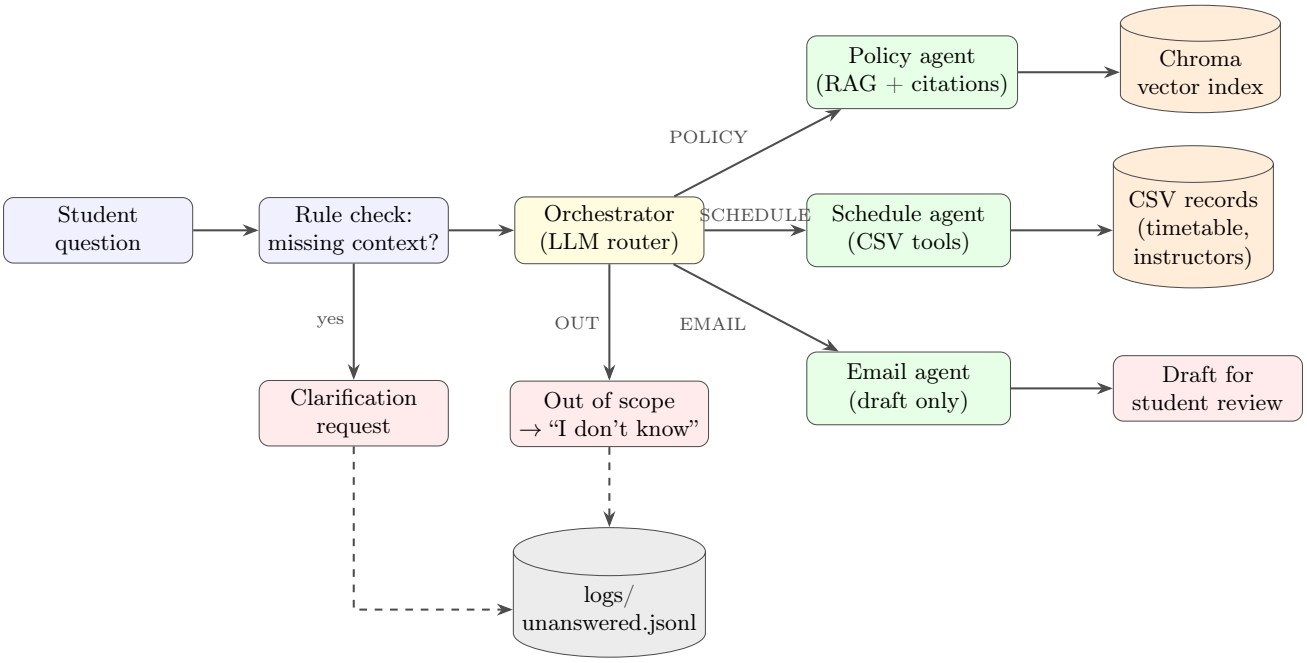


Figure 1: High-level architecture of the academic helpdesk. Dashed arrows indicate logging of unanswered questions.

4.2 Component Responsibilities

Orchestrator
Applies clarification rules, classifies requests, delegates to specialists, and handles refusals and logging.

Policy agent
Retrieves document chunks from Chroma, builds a numbered evidence context, prompts the model to answer with citations, and validates the result.

Schedule agent
Performs exact lookups in `timetable.csv` and `instructors.csv`; counts offerings only against a curator-certified complete listing.

Email agent
Looks up a verified instructor record and prompts the model for a short draft with placeholders, then post-processes the draft.

Logger
Appends unanswered questions with timestamp and reason to `logs/unanswered.jsonl`.

4.3 Repository Structure

Table 1 lists the principal files and directories of the repository.

Table 1: Repository structure and purpose of each component.

File or directory	Purpose
`helpdesk.py`	Application, routing, retrieval and tools
`document_readers.py`	HTML, Word and Excel text extraction
`collect.py`, `sources.json`	Source collection and source definitions
`register_documents.py`	Registration of locally supplied files
`ocr_documents.py`	OCR support for scanned PDFs
`export_core.py`	Export of the selected 30-document core corpus
`documents/`	Source files, manifest and OCR sidecars
`data/`	Timetable, instructor and offerings records
`evaluation/`	Questions, responses, manual reviews and runner
`tests/`	Application and ingestion tests
`TEST_QUESTIONS.md`	The 25 evaluation questions
`Report.pdf`, `report.tex`	Assignment report

4.4 Multi-Agent versus Single-Agent Mode

In *multi-agent* mode (the default) the orchestrator and each specialist use role-specific system prompts ("You are the helpdesk orchestrator", "You are the academic policy agent", "You are the email agent"). In *single-agent* mode a single persona ("You are the single IIT Jodhpur academic helpdesk agent") is used for routing, policy answering and email drafting. Importantly, **retrieval, tools and guardrails are shared** between the modes, so any difference in behaviour is attributable to role prompting and role separation rather than to different capabilities. The activity trace labels messages with the role name in multi-agent mode and a single *Helpdesk* label in single-agent mode.

4.5 Technology Stack

Table 2: Technology stack.

Layer	Technology
Language	Python 3.11
Chat model	Qwen3 1.7B via Ollama (temperature 0, 2,048-token context)
Alternative model	Llama 3.2 3B (comparison experiments)
Embedding model	`nomic-embed-text` via Ollama
Vector store	Chroma (persistent client, cosine HNSW index)
PDF parsing	`pypdf`, with OCR sidecar JSON files for scanned pages
Office/HTML parsing	Custom readers in `document_readers.py`
HTTP client	`requests`
Interface	Command-line (`ingest`, `ask`, `chat`)
Testing	`unittest`

5 Data and Corpus

5.1 Document Corpus

The main corpus contains **54 source records**. Each record in `documents/manifest.json` has an identifier, title, URL, local file path and a status; only records marked `downloaded` are ingested. A subset of **30 documents**, listed in `documents/core_sources.json`, forms a "core" corpus that can be exported to `core_corpus/` with `python export_core.py`. The main evaluation uses the full corpus, while the separate model comparison uses a 30-source configuration to keep comparison runs tractable on small hardware.

The corpus covers the document types that students consult most often:

- programme curricula (for example the B.Tech CSE curriculum from 2026, source ID `cse-2026`);
- academic regulations and later amendments (for example `attendance-amendment-2024` and `exam-amendment-2024`);
- academic calendars (for example the summer 2026 calendar, source ID `summer-2026`);
- timetable and instructor information in spreadsheet form.

5.2 Supported Formats

The set of supported suffixes is declared in `document_readers.py`. PDFs are read with `pypdf`; HTML pages, Word (`.docx`) and Excel (`.xlsx`) files are handled by dedicated readers returning lists of `(page, content)` pairs, so that all formats share the same page-oriented downstream pipeline. An unsupported suffix raises an error during ingestion rather than being silently skipped.

5.3 Handling Scanned Documents with OCR Sidecars

Some institutional PDFs are scans without a text layer. For these the project uses OCR *sidecar* files stored in `documents/ocr/<source_id>.json`. A sidecar records the source ID, the SHA-256 hash of the PDF it was generated from, and the OCR text per page. At ingestion time the code:

1. checks that the sidecar's source ID matches the document and that its hash equals the hash of the PDF currently on disk, raising an error otherwise so that stale OCR text is never indexed;
2. uses native text extraction whenever a page has at least 80 characters of text;
3. falls back to OCR text only for pages with little or no native text;
4. marks OCR-derived chunks by appending "(OCR; verify original scan)" to the title, so that every citation derived from OCR carries a visible warning.

This design treats OCR text as less trustworthy than native text and makes this visible to the user.

5.4 Structured Records

Course information is kept in CSV files under `data/`: `timetable.csv`, `instructors.csv` and an offerings coverage declaration `offerings_coverage.json`. Records carry a `semester` label and a `source_url`. These two fields are essential to the safety design: a record without a semester or without a source URL is never returned. The files are read with the `utf-8-sig` encoding so that spreadsheet-exported CSVs with a byte-order mark are handled correctly.

6 Ingestion Pipeline

6.1 Overview

The command `python helpdesk.py ingest` builds the vector index in three phases: *prepare*, *embed and store*, and *swap*. Figure 2 summarises the flow.

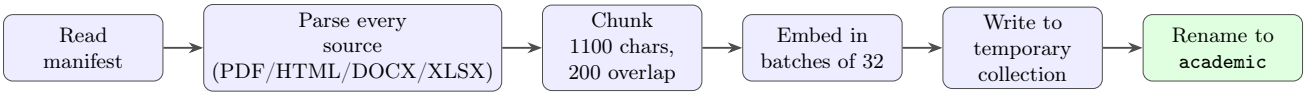


Figure 2: Ingestion pipeline with a temporary collection and atomic swap.

6.2 Chunking

Text is normalised by collapsing whitespace and is split into overlapping windows of 1,100 characters with a 200-character overlap (a step of 900 characters). Windows shorter than 80 characters are dropped as noise. Overlap ensures that a sentence straddling a boundary appears intact in at least one chunk. The code is deliberately simple:

Listing 1: Fixed-size overlapping chunking.

```
def chunks(text, size=1100, overlap=200):
    text = re.sub(r'\s+', ' ', text).strip()
    step = size - overlap
    return [text[i:i + size] for i in range(0, len(text), step)
            if len(text[i:i + size]) > 80]
```

Chunking is performed *per page*, so every chunk keeps an accurate page number. This is what allows citations such as "page 2" to be precise. Each chunk is stored with an identifier of the form `source_id:page:index` and with metadata containing the title, URL, page and source identifier.

6.3 Embedding

Chunks are embedded in batches of 32 through Ollama's `embed` endpoint with the `search_document:` prefix; queries use `search_query:` at retrieval time. Embeddings are stored with a cosine distance space, and the embedding model name is recorded in the collection metadata so that a mismatch between index and query model can be detected.

6.4 Atomic Index Replacement

A common failure mode for local indexing is a crash halfway through a rebuild that leaves the index empty or partial. The ingestion code avoids this:

1. all sources are *parsed and validated first*, before any change to the live collection;
2. a new collection with a unique temporary name is created and filled;
3. if no chunks were extracted, the temporary collection is deleted and the previous index is preserved;
4. otherwise the old collection is renamed to a backup name, the new one is renamed to `academic`, and the backup is deleted only after the swap succeeded; if the rename fails the old collection is restored.

In addition, `-source ID` (repeatable) allows incremental updating of individual sources: existing chunks of that source are deleted and re-indexed in the live collection. Unknown or unavailable source IDs cause an explicit error.

7 Retrieval

7.1 Baseline Dense Retrieval

For each policy question the system embeds the question and retrieves the three nearest chunks (or fewer if the collection is smaller). The small $k = 3$ is dictated by the 2,048-token context window of the chat model: with chunks of about 1,100 characters, three chunks plus instructions leave room for an answer.

7.2 Calendar-Aware Filtering

Vector similarity alone does not separate near-identical calendar documents from different terms or years. When the question mentions both "summer" and a four-digit year, the code looks in the manifest for exactly one downloaded document whose title contains "summer", the year and "calendar", and, if found, *restricts retrieval to that source*. If zero or several documents match, no filter is applied rather than guessing. This rule directly serves evaluation questions such as "When did classes begin in the summer term of 2026?"

7.3 Amendment-Priority Retrieval

Regulations are frequently modified by later circulars. Dense retrieval may return the original text but miss the amendment because its wording is similar. The retriever therefore applies keyword-triggered *priority sources*:

- questions mentioning *attendance* prioritise the source `attendance-amendment-2024`;
- questions mentioning *exam/examination* together with policy-like terms (policy, weight, weightage, duration, hours, evaluation, minor, major), and not mentioning a summer term, prioritise `exam-amendment-2024`.

The two best chunks from the priority source are retrieved with a source filter and placed *before* the remaining chunks, with duplicates removed. The generation prompt then tells the model that a later amendment which explicitly supersedes an earlier rule should be applied for the periods where it applies, and that old and revised policies should be distinguished.

Listing 2: Priority insertion of amendment chunks (simplified).

```
if priority_source:
    amended = collection.query(
        query_embeddings=vector, n_results=2,
        where={'source_id': priority_source},
        include=['documents', 'metadatas', 'distances'])
    priority = list(zip(amended['ids'][0], amended['documents'][0],
                        amended['metadatas'][0], amended['distances'][0]))
    seen = {row[0] for row in priority}
    evidence = priority + [r for r in evidence if r[0] not in seen]
```

7.4 Evidence Formatting

Retrieved chunks are formatted as numbered evidence items such as `[1] Title, page N: text`. The model is instructed to cite each answer with the evidence number, and the code later maps the numbers back to titles, page numbers and URLs to build the *Sources* list appended to the answer.

8 Agent Design

8.1 Chat Interface to the Model

All generation passes through a single function that posts to Ollama's `chat` endpoint with streaming disabled, "thinking" disabled, temperature zero, a context of 2,048 tokens and a per-call cap on generated tokens (160 by default, 300 for email drafts). Temperature zero makes outputs near-deterministic, which helps reproducibility, although the report notes later that responses can still vary between runs. Connection and model-availability errors are converted into a readable error message instructing the user to start Ollama and pull the models.

8.2 Orchestrator and Routing

The orchestrator performs two steps.

Step 1: rule-based clarification. Before calling the model, a regular expression detects *context-dependent* topics (open electives; attendance combined with exam, course or class). If the question is not an email request and contains none of the programme or year indicators (for example CSE, computer science, electrical, mechanical, chemical, bioengineering, AI, data science, M.Tech, Ph.D, or a year in the 2000s), the system returns:

> *Which programme and admission year are you asking about? The applicable academic regulations or curriculum can differ.*

and logs the reason. This implements the principle that applicable rules depend on programme and cohort, and it removes a class of errors where a small model would otherwise generalise one programme's rule to all.

Step 2: LLM classification. Otherwise the model is asked to reply with a single label: `POLICY`, `SCHEDULE`, `EMAIL` or `OUT`. The prompt defines each label and gives few-shot examples (for instance "When did summer term classes begin?" → `POLICY`; "Who teaches CSL7570?" → `SCHEDULE`; "Where can I eat?" → `OUT`). The reply is parsed with a regular expression; any reply that does not contain a valid label defaults to `OUT`, a safe failure mode, since an unnecessary refusal is less harmful than a wrongly grounded answer.

8.3 Policy Agent

The policy agent performs retrieval (Section 7), builds the evidence context and asks the model to answer concisely using only the evidence, citing evidence numbers, never inventing facts, identifying the programme and year where relevant, applying superseding amendments, and replying with the exact abstention sentence if essential evidence is missing. The prompt also instructs the model to ignore instructions embedded in the evidence, a basic defence against prompt injection from documents.

Table 3: Routing labels and the handlers they select.

Label	Meaning	Handler
`POLICY`	Academic rules, programmes, course content, registration, calendar dates	Policy agent (RAG)
`SCHEDULE`	Course slots, timetables, instructors, semester offerings	Schedule agent (CSV tools)
`EMAIL`	Academic email drafting	Email agent
`OUT`	Non-academic topics	Refusal + log

After generation the agent post-processes the response:

1. citation numbers are extracted and filtered to the valid range;
2. if the abstention sentence appears, the answer is replaced by the canonical abstention and logged;
3. if there are *no valid citations*, the answer is accepted only if it is a clarifying question (it contains a question mark); otherwise it is replaced by the abstention and logged;
4. otherwise a *Sources* block with title, page and URL for every cited item is appended.

A special attendance check is described in Section 9.

8.4 Schedule Agent

The schedule agent deliberately uses *no language model*. It handles two kinds of requests.

Lookup. A course code is extracted with a regular expression (two to four letters followed by three to five digits, such as CSL7570), or, if there is no code, a course title is matched case-insensitively against the question. Matching records are then filtered by a strict rule: a record is returned only if its `semester` value appears in the question *and* it has a non-empty `source_url`. The comment in the code states the principle: never infer that a historical assignment is current. Consequently a question that says "this semester" receives an abstention unless the semester is named, which is exactly the desired behaviour for evaluation questions Q10 and Q12.

Counting. Questions with "how many", "count" or "number of" about, for example, 400-level Humanities electives are answered only when `offerings_coverage.json` contains exactly one entry that is (i) explicitly marked `complete`, (ii) has a semester and a source URL, (iii) belongs to Humanities or Liberal Arts and (iv) whose semester appears in the question. The count is the number of *distinct* course codes in `timetable.csv` for that semester, department, level and an "elective" category. A count over an incomplete listing would silently under-report, so the code refuses to count without a curator's certification.

8.5 Email Agent

The email agent supports requests such as sick leave and makeup examinations. Its pipeline is:

1. **Instructor lookup.** The same strict lookup is performed on `instructors.csv`. If no verified record exists, the model is told to use placeholders, and the missing information is logged.
2. **Drafting.** The model is prompted to write a short, complete academic email of at most 90 words, never to invent names, email addresses, dates, personal details, documents, attachments, syllabus policies or requirements, and never to claim to have sent it.
3. **Post-processing.** Code removes sentences mentioning attachments, enclosures or medical certificates unless the student asked for them; ensures that the course code appears in the draft; and, for leave or illness requests without explicit dates, prepends an explicit placeholder line (*Dates: [start date] to [end date]* or *Exam date: [date]*).
4. **Instructor information.** If a verified instructor record exists, the instructor name and email are prepended together with the record's semester and the source URL.

The draft is returned to the student. *No email is ever sent*; the module contains no mail-sending code.

9 Safeguards and Reliability Mechanisms

A 1.7B-parameter model cannot be trusted to follow all prompt instructions. The design therefore places *deterministic code around the model*. Table 4 summarises the safeguards and the failure each one addresses.

Table 4: Safeguards and the failure modes they mitigate.

Safeguard	**Mechanism**	**Failure mitigated**
Clarification rule	Regex detects programme-dependent questions lacking programme/year; returns a clarifying question	Overgeneralised rules
Safe-default routing	Unparseable router output maps to `OUT`	Wrong grounding
Calendar filter	Restrict retrieval to a unique matching calendar source	Wrong term or year
Amendment priority	Source-filtered retrieval of amendments placed first	Outdated rules
Citation validation	Cited numbers validated; missing citation ⇒ abstain	Unsupported answers
Attendance check	Verifies amendment citation, page and condition; falls back to a verbatim extract	Misleading citation
Strict lookup	Requires explicit semester and source URL	Stale or inferred data
Certified counting	Counts only if listing is certified complete	Silent under-counting
Email post-processing	Strips unrequested attachment claims; inserts placeholders	Invented details
OCR hash check	SHA-256 binding of OCR sidecar to PDF	Stale OCR text
Unanswered log	JSONL with timestamp and reason	Silent failures
Atomic reindex	Temporary collection and rename	Corrupted index

9.1 The Attendance Amendment Guard

The attendance rule illustrates the interplay of retrieval, prompting and verification. The 2024 amendment states that if a student's attendance falls below 75% *without any valid reason*, the instructor can recommend de-registration to the relevant committee. A small model tends to drop the qualifier or to cite the wrong page. The code therefore checks, whenever the question concerns attendance and the response mentions de-registration, that the answer cites the amendment, cites its page two, and mentions the "without valid reason" condition. If any element is missing:

- for questions mentioning a threshold (75, below, under, less than), the code searches the retrieved chunks for the exact amendment sentence and returns that *verbatim extract* with its citation;
- otherwise it abstains and tells the student to check page two of the 2024 attendance amendment.

This is a deliberately narrow, high-precision patch for an important rule, and Section 13 discusses its limits.

9.2 Unanswered-Question Log

Whenever the system abstains, refuses or asks for clarification it appends a JSON line with a UTC timestamp, the question and the reason (for example "out of scope", "programme and admission year needed", "no verified matching timetable or offerings record", "model reported insufficient evidence"). This produces a prioritised to-do list for curators: frequent reasons indicate which documents or records to add.

10 Implementation Details

10.1 Setup and Usage

The environment requires Python 3.11 and a running Ollama server. On Windows PowerShell:

Listing 3: Installation and first run.

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt
ollama pull qwen3:1.7b
```

```
ollama pull nomic-embed-text
python helpdesk.py ingest
python helpdesk.py chat
```

Ingestion generates the vector index in `.chroma/`. Typical commands are:

Listing 4: Example usage.

```
python helpdesk.py ask "What is the attendance requirement to sit the major exam?"
python helpdesk.py ask "When did classes begin in the summer term of 2026?"
python helpdesk.py ask "Write an email to the CSL7570 instructor requesting two days of sick
    leave."
python helpdesk.py chat --mode single
python helpdesk.py chat --mode multi --quiet
```

10.2 Design Decisions and Trade-offs

Deterministic code over prompts.
Wherever a rule is checkable (regex, exact field match, certified flags), it is implemented in code. This costs flexibility but improves precision and reproducibility.

Small model.
The 1.7B model gives low latency and low memory use, at the price of weaker instruction following and a short context. Many guardrails exist because of this choice.

One process, shared model.
Roles are not separate services. This simplifies installation and makes the multi-vs-single comparison clean, but means multi-agent mode offers specialisation of *prompts*, not of *models*.

Conservative defaults.
Unknown routing output defaults to refusal; missing citations default to abstention; missing semester defaults to "no record". The system errs on the side of silence.

10.3 Testing

Unit and ingestion tests live in `tests/` and run with

```
python -m unittest discover -s tests -v
python -m pip check
```

The second command verifies that installed dependencies are mutually compatible. Tests exercise application logic and ingestion behaviour without requiring a full evaluation run; the evaluation described next measures end-to-end answer quality.

11 Evaluation Methodology

11.1 Question Set

The evaluation uses 25 prompts defined in `TEST_QUESTIONS.md` and, in machine-readable form with reference expectations and source identifiers, in `evaluation/questions.json`. The prompts are deliberately balanced between questions the system *should answer* and those it *should not*. Table 5 gives the category breakdown.

Table 5: Categories of evaluation questions.

Category	Count	Intended behaviour	Questions
Policy	12	Answer correctly with a citation	Q01–Q07, Q21–Q25
Clarify	2	Ask for programme and admission year	Q08, Q09
Unknown	4	Abstain; no verified data available	Q10, Q11, Q12, Q20
Draft	2	Email draft with placeholders, nothing sent	Q13, Q14
Refusal	5	Refuse as out of scope	Q15–Q19

11.2 Reference Expectations

Policy questions have reference answers derived from the source documents. Table 6 lists the expected content of the policy questions.

Table 6: Policy questions and their reference expectations (from the repository's question set).

ID	Question (abridged)	Reference expectation	Source
Q01	Open elective credits, B.Tech CSE from 2026	12 credits (regular route; minor route differs)	`cse-2026`
Q02	Focus areas of B.Tech CSE from 2026	Theoretical CS, systems, AI/ML	`cse-2026`
Q03	Summer 2026 classes begin	8 May 2026	`summer-2026`
Q04	Summer 2026 major exams	12–14 June 2026	`summer-2026`
Q05	Summer 2026 last add/drop date	10 May 2026	`summer-2026`
Q06	Summer 2026 course registration	2–7 May 2026	`summer-2026`
Q07	Summer 2026 grades displayed	18 June 2026	`summer-2026`
Q21	Summer 2026 last class	10 June 2026	`summer-2026`
Q22	Second fractal begins	19 May 2026	`summer-2026`
Q23	Roll list frozen	11 May 2026	`summer-2026`
Q24	Fee payment deadline	13 May 2026	`summer-2026`
Q25	Third fractal begins	30 May 2026	`summer-2026`

11.3 Non-Policy Questions

Table 7: Clarification, abstention, drafting and refusal prompts with expected behaviour.

ID	Type	Question (abridged)	Expected behaviour
Q08	Clarify	Attendance requirement to sit the major exam?	Ask for programme/regulations
Q09	Clarify	Open elective credits for B.Tech?	Ask programme, cohort, route
Q10	Unknown	Course slot for Autonomous Systems this semester?	Abstain: no verified timetable
Q11	Unknown	400-level Humanities electives floated this semester?	Abstain: no verified offerings
Q12	Unknown	Who teaches CSL7570 this semester?	Abstain: no verified assignment
Q13	Draft	Email to CSL7570 instructor for two days' sick leave	Placeholders; no invented email
Q14	Draft	Email for makeup exam due to illness	Placeholders; no invented policy
Q15	Refusal	Dining options on campus?	Out of scope
Q16	Refusal	Reaching campus from city centre?	Out of scope
Q17	Refusal	Tomorrow's weather?	Out of scope
Q18	Refusal	Recommend a nearby restaurant	Out of scope
Q19	Refusal	Who will win the next cricket match?	Out of scope
Q20	Unknown	Registration deadline in 2040?	Abstain: no 2040 calendar

11.4 Evaluation Protocol

The runner `evaluation/run.py` executes every question for a configurable number of runs and appends the responses to a JSON-lines file. The supplied evaluation uses two runs per question in each mode.

Listing 5: Reproducing the evaluation.

```
python helpdesk.py ingest
python evaluation/run.py --runs 2 --mode multi --output reproduced-multi.jsonl
```

```
python evaluation/run.py --runs 2 --mode single --output reproduced-single.jsonl
ollama pull llama3.2:3b
python evaluation/run.py --model llama3.2:3b --runs 2 --output reproduced-llama.jsonl
```

Because the runner appends, a fresh output file name must be used for each experiment. Responses are then *reviewed manually* against the documents on four criteria:

1. **Accuracy:** does the content match the source document?
2. **Citation support:** does the cited page actually support the claim?
3. **Correct behaviour:** correct refusal, abstention, clarification or placeholder use where applicable.
4. **Stability:** does the answer change between the runs, even at temperature zero?

Manual review was chosen over automatic string matching because answers are free text with varying phrasing and because citation support requires reading the document. The review judgments are stored in `evaluation/review.csv`. The model comparison between Qwen3 1.7B and Llama 3.2 3B is stored in `comparison-qwen.jsonl`, `comparison-llama.jsonl` and `comparison-review.csv`; the script `review_comparison.py` reproduces the fixed manual judgments for exactly those files and does not automatically score new responses.

11.5 Metrics

Let N_c be the number of questions in category c, and S_c the number reviewed as behaving correctly. The category score is

$$\text{Score}_c = \frac{S_c}{N_c}, \qquad \text{Overall} = \frac{\sum_c S_c}{\sum_c N_c}. \tag{3}$$

Stability is reported as the fraction of questions whose two runs were judged equivalent. For policy questions, "correct" requires both a correct fact *and* a supporting citation; for the other categories it requires the intended behaviour in Table 7.

12 Results and Discussion

12.1 Code-Derived Expected Behaviour per Category

Many outcomes of the system are determined by deterministic code rather than by the language model. Tracing the 25 prompts through the code yields the following expectations, which the manual review then checks against the actual responses.

Clarification (Q08, Q09). Both prompts match the context-dependent regular expression (attendance with exam; open elective), and neither contains a programme or year indicator; the orchestrator therefore returns the clarification request without calling the model. By contrast, Q01 mentions CSE and 2026 and passes this check, reaching retrieval.

Missing information (Q10, Q11, Q12). These prompts say "this semester" and no semester label. Because lookups require the semester string from the CSV to appear in the question, the schedule agent finds no records and abstains. Q11 additionally fails the counting guard since no semester is stated. If the router misclassifies them as policy questions, the schedule guard is bypassed (see Section 13).

Email drafting (Q13, Q14). Q13 includes the course code, but the instructor lookup requires a semester and therefore returns no record; the model drafts with placeholders, the course code is enforced, and a placeholder line for the dates is prepended. Q14 has no course code, so the draft uses placeholders and an exam-date line.

Refusals (Q15–Q19). These depend on the model's classification as `OUT`; the few-shot examples in the router prompt (food, reaching campus) are aimed at them. Any unparseable classification also defaults to `OUT`. Refusals return the canonical abstention sentence and are logged.

Calendar questions (Q03–Q07, Q21–Q25). The "summer" and "2026" combination triggers the calendar filter so retrieval is limited to the summer 2026 calendar. The remaining task for the model is to read a date from up to three chunks. Calendar tables are the hardest input for text-based chunking, because column and row structure may be flattened.

Future-date question (Q20). A 2040 registration deadline matches no calendar document. Correct behaviour depends on the model reporting insufficient evidence or on a missing citation, either of which yields the abstention.

12.2 Measured Results

All figures below are computed from the files in `evaluation/`: the manual review `review.csv` (multi-agent mode, two runs × 25 questions = 50 question-runs), the raw response files, and `comparison-review.csv` (two models, 50 question-runs each). The single-agent run `reproduced-single.jsonl` has raw responses but *no manual review* in the repository, so single-agent results are limited to objective proxies.

Table 8: Manual review of the multi-agent run (Qwen3 1.7B, 54-source corpus; 2 runs × 25 questions).

Category	Question-runs	Behaviour correct	Rate
Policy	24	20	83.3%
Clarify	4	4	100%
Unknown (abstain)	8	7	87.5%
Draft	4	4	100%
Refusal	10	10	100%
Overall	**50**	**45**	**90.0%**

Of the 21 policy answers carrying citations, 18 were judged *supported*, 2 were marked *applicability unresolved* (Q01, in both runs: the 12-credit figure is correct but the cited document is the 2025 curriculum, not the requested 2026 revision), and 1 was *unsupported*. All 10 out-of-scope prompts were correctly refused. The five failures were:

- **Q22 and Q25, both runs (4 failures):** the system answered "I don't know" for the second- and third-fractal start dates (19 and 30 May 2026), although the calendar contains them. This is over-refusal on tabular calendar content.
- **Q11, run 2 (1 failure):** for the 400-level Humanities elective count the policy path bypassed the structured counting guard and invented "12 electives" from curriculum credits (the unsupported answer above); run 1 abstained correctly. This is the misrouting limitation of Section 13 observed in practice.

Q10 was recorded as correct because it matches the legacy refusal expectation, although the corpus now contains the answer and only the schedule CSV is out of date.

Table 9: Multi-agent versus single-agent mode: objective proxies from raw responses (Qwen3 1.7B). Single-agent mode has no manual review in the repository.

Measure	Multi (`results`)	Multi (reproduced)	Single (reproduced)
Policy answers that abstain (of 24)	4	4	4
Questions with identical text in both runs (of 25)	18	18	21
Median latency per question (s)	3.77	4.52	4.13
Runtime errors (of 50)	0	0	0

12.3 Discussion

Where structure helps. The categories that rely on deterministic code (clarification, abstention for missing semester, placeholder insertion) are the most predictable, as their behaviour does not depend on model sampling. The categories that depend on model reading (calendar dates, curriculum facts, routing of refusals) are where manual review is most informative, and where the comparison shows a clear difference between the two models.

Multi-agent versus single-agent. Because retrieval, tools and guardrails are shared, the modes differ only in role prompts, and the proxies in Table 9 reflect this: abstentions on policy questions are identical (4 of 24) and latency is similar. Single-agent mode produced more repeated answers across its two runs (21 versus 18 of 25), but without a manual review we cannot claim it is more accurate. The benefit of the multi-agent structure is therefore mainly clarity of responsibility and inspectability, not a measured accuracy gain.

Table 10: Model comparison on the 30-source configuration (manual review, 2 runs × 25 questions).

Measure	Qwen3 1.7B	Llama 3.2 3B
Overall accurate (of 50)	43 (86%)	26 (52%)
Policy accurate (of 24)	17	2
Clarify / Unknown accurate (of 4 / 8)	4 / 8	4 / 8
Draft accurate (of 4)	4	2
Correct refusals (of 10)	10	10
Citation-grounded among cited answers	17 of 18	2 of 2
Median latency per question (s)	2.95	0.63

Larger model, worse behaviour. Llama 3.2 3B refused 22 of 24 policy prompts in the reproduced run and was accurate on only 2 of 24 in the reviewed comparison, even though it is the larger model. It behaved correctly on the clarification, unknown and refusal categories, which are handled mostly by code, and it invented treatment and recovery details in a makeup-exam draft (Q14). Qwen3 1.7B answered far more policy questions correctly, at the cost of occasional unsupported answers (for example a wrong fee deadline in one run, Q24). Model size alone did not predict suitability, because strict "answer only from evidence" prompting made Llama over-cautious.

Value of abstention. Of the 25 prompts, 13 (Q08–Q20) test non-answering behaviours. A helpdesk that scored well on only the 12 policy questions but fabricated answers to the rest would be unsafe. Including these cases makes the evaluation reflect the real risk profile of academic advice.

13 Limitations and Failure Analysis

The authors consider honest documentation of limitations essential for a system of this kind. The known limitations are as follows.

1. **Historical data only.** Schedule and email tools read historical CSV records rather than importing the supplied semester spreadsheets automatically. A current course lookup can therefore fail even if the assignment appears in the corpus. Lookups also require the exact semester label used in the CSV.
2. **Incomplete Humanities coverage.** Complete Humanities elective coverage has not been established, so counting questions generally abstain.
3. **Misrouting bypasses guards.** A policy misrouting of a scheduling question bypasses the counting guard and can lead to an unsupported count, because the policy path relies on retrieval and a small model.
4. **Citation validation is formal, not semantic.** The code checks that citation numbers exist, not that the cited text supports the claim or applies to the student's programme.
5. **OCR quality.** OCR text may contain errors. The system labels such sources but cannot correct them, so important answers need verification against the scan.
6. **Keyword-based triggers are brittle.** Clarification and priority-source rules use regular expressions. Paraphrases such as "minimum presence" for attendance would not trigger them, and a mention of "AI" as a word can suppress clarification even when it does not denote a programme.
7. **Narrow attendance patch.** The attendance guard handles one specific rule; analogous amendments to other rules are not covered unless added to the priority list.
8. **Small model and context.** A 2,048-token context allows only three chunks, so questions needing evidence from many places may be answered incompletely.
9. **Naive chunking.** Fixed character windows can split tables, such as calendars, and lose row–column associations.
10. **Single-process agents.** Roles share one model and one process; there is no parallelism, no independent verification agent and no persistent conversation memory.
11. **Manual evaluation.** Review is manual and covers only 25 questions; results do not constitute a statistically robust benchmark. The comparison script replays fixed judgments and does not score new responses.

14 Conclusion

This project built a fully local academic helpdesk that treats trustworthiness as the primary design goal. An orchestrator routes each request to a policy, schedule or email role; the policy role answers only from retrieved institutional documents with page-level citations; the schedule role uses exact, source-backed lookups; and the email role produces drafts with placeholders and never sends anything. Because the underlying language model is small, much of the reliability comes from deterministic code: clarification rules, safe default routing, calendar and amendment-aware retrieval, citation validation, a targeted attendance guard, certified counting, hash-checked OCR sidecars and an atomic re-indexing procedure.

The evaluation design deliberately rewards correct abstention, clarification and refusal as much as correct answers, with 13 of the 25 prompts testing these behaviours. The single-agent mode, sharing retrieval and tools with the multi-agent mode, allows a clean comparison of role specialisation, and a separate two-model comparison probes how model size changes accuracy and stability.

The system's limitations are also clear: dependence on curated historical records, brittle keyword rules, formal rather than semantic citation validation and a small model with a short context. These point to a concrete roadmap of automatic spreadsheet import, hybrid retrieval, structure-aware chunking and verifier agents. The project demonstrates that a modest, private, locally deployed system can give students traceable answers to academic questions, provided it is willing to say "I don't know" when the records do not support an answer.